CYBERKINKS
Lougie Caday
Justine Aaron Runes
Eric Lor
Jayson Huub M. Partido

Create a synthesis from the lecture notes and the supplemental videos that you watched about “OS memory management: process” and “OS2 memory manager”

Operating System #05 Memory Management: Process, Fragmentation, Deallocation,

The operating system (OS) plays a crucial role in managing the computer's hardware, including its memory, which is often considered the second most important resource after the CPU. Memory management is essential because it ensures that programs can run efficiently while utilizing the limited Random Access Memory (RAM) available. This synthesis explains how the OS manages memory, supports multiple processes, and addresses challenges such as fragmentation.

When a program is compiled, it becomes an executable file stored on the hard drive. When a user runs the program, it creates a "process," which is loaded into RAM to execute. Each process has segments in memory, such as the text segment (code instructions), the stack (temporary data), and the heap (dynamic memory). However, since RAM is limited (4GB, 8GB, or 16GB), the OS must ensure that multiple processes can execute either simultaneously or in a time-sliced manner.

Initially, older systems used a **single contiguous model**, where only one process could occupy the entire RAM. While simple, this model was inefficient because no other process could run until the current one finished. Additionally, the process size was limited to the RAM's total size, making it unsuitable for large processes or multitasking.

To improve this, systems adopted the **partition model**, which divides RAM into smaller segments, allowing multiple processes to coexist. Each partition has a base address, size, and a status indicating whether it is in use. This model allows multiple processes to run in parallel or sequentially, depending on the CPU scheduling. However, managing multiple partitions introduces challenges like **fragmentation**. Fragmentation occurs when free memory is scattered in small, non-contiguous blocks, making it impossible to load a process larger than any individual block, even if the total free memory is sufficient.

To handle fragmentation, the OS uses algorithms like **First Fit**, which assigns a process to the first available memory block that is large enough. While this strategy is simple, it may not always utilize memory optimally. Other algorithms, such as Best Fit or Worst Fit, aim to improve memory allocation efficiency, though they may involve additional complexity.

In conclusion, memory management is a fundamental responsibility of the operating system, ensuring that processes can run smoothly within the constraints of limited RAM. From early models like single contiguous memory to more advanced partitioning techniques, the OS continuously evolves to address challenges like fragmentation and to maximize memory

utilization. This balance of simplicity and efficiency allows modern systems to support multitasking and complex applications effectively.

Operating Systems 2 - Memory Manager

Synthesis Essay: The Evolution of Memory Management in Operating Systems

Memory management is a fundamental component of operating systems, ensuring that a computer can efficiently allocate and use its memory. Over the years, advancements in memory management techniques have been crucial to the development of more powerful and efficient computing systems. From early methods like single-user contiguous memory allocation to the modern approach of virtual memory, the evolution of these techniques reveals both the challenges and solutions in managing a computer's memory. This essay synthesizes key concepts and methods in memory management, exploring their progression and how they have shaped the way operating systems handle data today.

In the early days of computing, the most basic form of memory management was the **single-user contiguous allocation**. This method assigned all available memory to a single job, processing it sequentially. While simple, this approach wasted memory and was inefficient, especially when jobs were smaller than the available memory or when the job was too large to fit. As a result, the need for more efficient allocation systems emerged. This led to the introduction of fixed partitioning, where memory was divided into fixed-sized sections. Each job would be loaded into one of these partitions, allowing multiple jobs to run simultaneously. However, fixed partitions still had limitations. The fixed sizes of partitions led to fragmentation, where memory remained unused because jobs couldn't perfectly fit into the pre-allocated sections.

The next significant advancement was **dynamic partitioning**, which solved the issue of fixed partition sizes by allocating memory based on the actual needs of the job. This allowed for better memory utilization. However, dynamic partitions also introduced a new challenge: fragmentation. When jobs were completed and removed from memory, gaps could form, leading to inefficient memory usage. To further improve memory allocation, operating systems implemented various partition allocation strategies such as first-fit and best-fit, which determined how to allocate memory for new jobs. Despite these improvements, the problem of memory fragmentation remained, necessitating a more sophisticated solution.

Virtual memory, which emerged as the next step in memory management, revolutionized the way operating systems handle memory. Rather than requiring that entire programs be loaded into contiguous blocks of memory, virtual memory allows programs to be divided into smaller, manageable pages that can be loaded and unloaded dynamically. This method, known as paging, ensures that only the parts of a program that are actively needed are kept in memory, reducing wasted space. However, managing these pages requires additional tables, such as the page map table and the memory map table, to track the locations and statuses of each page. Though this increases complexity, it significantly improves memory efficiency.

Demand paging further enhanced virtual memory by loading pages only when they are required, rather than preloading them. This means that if a program needs to access a page that's not currently in memory, the operating system will load it on demand. The key challenge here lies in managing which pages remain in memory when new ones need to be loaded. Various page replacement algorithms, such as first-in, first-out (FIFO) and least recently used (LRU), help address this issue, ensuring that the most important pages are available to the program at the right time.

Another advancement in memory management was **segmented memory allocation**, which broke a program into segments that reflected its logical structure, such as functions or data blocks. This method allowed for better organization of a program's memory requirements. By splitting the program into logical units, the operating system could handle different parts of the program more efficiently. The drawback of segmented memory allocation, however, is that it could still lead to fragmentation, especially when segments are of different sizes.

Modern operating systems often combine **paging and segmentation**, creating a hybrid approach to memory management. This combination allows the operating system to take advantage of the benefits of both methods: the flexibility of segmentation and the efficiency of paging. Hybrid systems track segments using a segment map table and pages through a page map table, offering a more sophisticated solution to memory management.

Virtual memory is perhaps the most significant advancement in memory management, as it allows systems to expand beyond their physical memory limitations. By using secondary storage, such as hard drives, to temporarily hold data, virtual memory enables systems to handle larger programs and multiple processes simultaneously. Although secondary storage is slower than RAM, it provides an essential buffer for operating systems when RAM is full. This ability to extend memory capacity has made modern computing more efficient and capable of handling complex tasks.